

Controla.AI — Documentação TCC

Documento único e oficial do Trabalho de Conclusão de Curso.
Descreve arquitetura, lógica de negócio, banco de dados e fluxos do sistema.
Regra de manutenção: qualquer alteração de código, schema, rotas ou pastas **deve ser refletida aqui** na mesma entrega.

Versão: 5.0 · Última revisão: jun/2026 · Repositório: Controla.AI

Guia rápido — Onde está cada módulo

Módulo	Pasta / arquivos principais
Integração WhatsApp (Baileys)	backend/whatsapp/ — client.ts, message-handler.ts, whatsapp-bubbles.ts, user-resolver.ts, routes.ts
Consultas financeiras	backend/api/insights.ts — respostas a "quanto gastei?", projeções, relatórios e KPIs
Integração IA com Baileys	backend/whatsapp/message-handler.ts → backend/api/financial-agent.ts, parser.ts, openai-client.ts, media-processor.ts

Arquitetura do banco (MD + PNG): [./CONEXOES_BANCO_DADOS.md](#) (ligações claras) · [ARQUITETURA_BANCO_COMPLETA.md](#) (colunas + ER) · PNGs diagrama e detalhes.

Detalhamento completo: [§3.1 Mapa detalhado dos módulos backend](#).

Convenção de comentários no código (TCC)

O quê	Onde	Formato
Documento oficial	TCC_DOCUMENTACAO.md (raiz)	Arquitetura, fluxos, banco, mapa de arquivos
Índice backend	backend/src/MAPA-SISTEMA.ts	Catálogo + BACKEND_APPLICATION_FILES
Índice frontend	frontend/src/MAPA-SISTEMA.tsx	Catálogo + FRONTEND_APPLICATION_FILES
Código comentado	Arquivos em BACKEND_APPLICATION_FILES e FRONTEND_APPLICATION_FILES	Cabeçalho Doc TCC: TCC_DOCUMENTACAO.md + comentários inline em português
Excluídos	frontend/src/components/ui/*	Biblioteca shadcn (terceiros)
Excluídos	backend/src/db/seed*.ts	Scripts de demonstração

Regra: ao alterar código, atualizar `TCC_DOCUMENTACAO.md` (§14 histórico) na mesma entrega.

Índice

- 1. [Visão geral](#)
- 2. [Arquitetura do sistema](#)
- 3. [Estrutura de pastas](#)
- 4. [Fluxos principais](#)
- 5. [Backend — servidor \(src/\)](#)
- 6. [OpenAI \(api/\)](#)
- 7. [WhatsApp / Baileys \(whatsapp/\)](#)
- 8. [Frontend \(frontend/\)](#)
- 9. [Arquitetura de banco de dados](#)
- 10. [Autenticação e segurança](#)
- 11. [Variáveis de ambiente](#)
- 12. [Deploy e execução](#)
- 13. [Mapa de arquivos comentados](#)
- 14. [Histórico de alterações](#)
- 15. [Referência completa — agente IA, módulos e lógicas](#)

1. Visão geral

O **Controla.AI** é um sistema de controle financeiro pessoal que combina:

Camada	Tecnologia	Função
Frontend	React + Vite + Tailwind + shadcn/ui	Painel web, chat IA, metas, admin
Backend	Node.js + Fastify + TypeScript	API REST, JWT, orquestração
Banco	PostgreSQL (Railway / Neon) + Drizzle ORM	Persistência relacional
IA	OpenAI (GPT-4o, Whisper, visão)	Parser de linguagem natural, chat, OCR
WhatsApp	Baileys (@whiskeysockets/baileys)	Canal de entrada via mensagens

Problema resolvido: o usuário registra gastos e receitas por WhatsApp (texto, áudio, foto, PDF) ou pelo painel web; a IA interpreta a mensagem, categoriza e persiste no banco; o dashboard exibe KPIs, metas e relatórios.

2. Arquitetura do sistema

[Diagrama Mermaid — ver PNGs em ./png/ e ./DATABASE_DIAGRAMAS.md.]

Camadas lógicas

Camada	Responsabilidade
Apresentação	React SPA — login, dashboard, metas, chat IA, admin WhatsApp
API REST	Fastify — validação Zod, JWT, CRUD transações/metasp>
Domínio financeiro	Parser IA, categorização, metas, orçamento
Integração WhatsApp	Baileys — QR, sessão, mensagens, keep-alive
Integração OpenAI	GPT (parser/chat), Whisper (áudio), visão (notas)
Persistência	Drizzle ORM → PostgreSQL

3. Estrutura de pastas

```
controlaaii/
├─ TCC_DOCUMENTACAO.md      ← ESTE ARQUIVO (fonte única de verdade)
├─ ./                        ← PDF, PNGs ERD, snapshot do banco (entrega TCC)
│   ├── TCC_DOCUMENTACAO.pdf
│   ├── TCC_DOCUMENTACAO.md
│   ├── TCC_DOCUMENTACAO.txt  ← Versão texto plano
│   ├── DATABASE_DIAGRAMAS.md ← Índice com todos os PNGs
│   ├── DATABASE_SNAPSHOT.md
│   └─ png/                  ← Diagramas legíveis (por domínio e por tabela)
│       ├── 00-visao-geral.png
│       ├── grupo-core.png
│       ├── grupo-metas.png
│       ├── grupo-whatsapp.png
│       ├── grupo-ia.png
│       ├── grupo-outros.png
│       └─ tabela-<nome>.png  ← Uma tabela por PNG (todas as colunas)
├─ frontend/                ← SPA React
└─ backend/
    ├── .baileys-session/    ← Credenciais WhatsApp (NÃO versionar)
    ├── .controlaai/         ← runtime.json (modelo OpenAI escolhido pelo admin)
    ├── api/                 ← Módulo OpenAI + entry Vercel
    ├── whatsapp/            ← Módulo Baileys (conexão + mensagens)
    ├── src/                 ← Servidor core (Fastify, auth, banco)
    │   ├── index.ts         ← Boot do servidor
    │   ├── auth.ts          ← JWT
    │   ├── api-routes.ts    ← CRUD transações/categorias
    │   ├── extended-routes.ts ← Chat IA, KPIs, metas, imports
    │   ├── goals-service.ts  ← Progresso de metas
    │   ├── env.ts           ← Variáveis de ambiente
    │   ├── db/              ← Schema Drizzle + seeds
    │   └─ utils/            ← phone, money, admin
    ├── drizzle/             ← Migrations SQL
    └─ scripts/              ← Utilitários de banco
```

Princípio TCC: poucas pastas, arquivos únicos e comentados linha a linha em português.

3.1 Mapa detalhado dos módulos backend

Pasta backend/whatsapp/ — Integração WhatsApp (Baileys)

Responsável por **conectar o número oficial**, receber mensagens, enviar respostas em bolhas e chamar o agente IA.

Arquivo	Função detalhada
client.ts	Socket @whiskeysockets/baileys: QR code, reconexão, envio de texto, anti-replay (ignora mensagens >4 min e histórico nos 20s pós-conexão)
message-handler.ts	Pipeline principal — inbound → identifica usuário → mídia (áudio/imagem/PDF) → processFinancialAgentMessage → bolhas de resposta
whatsapp-bubbles.ts	Divide respostas longas () em várias mensagens WhatsApp (conversa humanizada)
user-resolver.ts	Telefone BR → users.id; cria usuário automático se necessário; variantes com/sem 9º dígito
jid-resolver.ts	Resolve JID LID/PN do Baileys; lê lid-mapping_*_reverse.json da sessão
inbound-reply-guard.ts	Garante que o bot só responde dentro de um inbound real (sem spam proativo)
message-dedup.ts	Deduplica por whatsappMessageId — evita processar replay na reconexão
routes.ts	API admin: GET/POST /api/admin/whatsapp/* (status, connect, disconnect, logs)
session-utils.ts	Caminho da pasta .baileys-session/ (credenciais locais)
keep-alive.ts	Timer 30 min — verifica saúde do socket e reconecta se cair
baileys-log.ts	Buffer circular (500 linhas) de logs Baileys para o painel admin

Sessão: backend/.baileys-session/creds.json (não versionar). Estado espelhado em whatsapp_connection (id=main).

Pasta backend/api/ — Agente IA, parser OpenAI e consultas

Orquestra linguagem natural → ações no banco. Usado pelo **WhatsApp** (message-handler.ts) e pelo **chat web** (extended-routes.ts).

Orquestração (WhatsApp + web)

Arquivo	Função detalhada
financial-agent.ts	Pipeline unificado — saudação, onboarding, renda, metas, parser, transações, consultas, fallback welcome
onboarding-agent.ts	Perfil de renda mensal: salva uma vez em budgets + defaults; não repete perguntas após 1º save
goal-agent.ts	Fluxo conversacional de metas (valor, prazo, INSERT goals)
goal-parser.ts	Extraí limit_amount, duration_months, deadline_at de texto livre
income-classifier.ts	Separa renda mensal vs ganho pontual ; sessão de clarificação 1/2
conversation-context.ts	Fase da conversa (goals/expenses), flag pós-registro (RAM)
conversation-history.ts	Histórico outbound + contexto para parser (10 msgs); anti-repetição
message-text.ts	Normalização de texto inbound; detecção de saudações e ajuda
transaction-intent.ts	Regex: isTransactionMessage, isExpenseMessage, isQueryMessage
assistant-response.ts	Templates ricos pós-transação e pós-registro
app-links.ts	URLs do painel Vercel; bolhas de boas-vindas e rodapés

Integração OpenAI (parser + mídia)

Arquivo	Função detalhada
parser.ts	parseFinancialIntent — GPT-4o-mini → JSON FinancialIntent; fallback regex local
prompts.ts	System prompts oficiais (parser, chat, visão, documentos PDF)
openai-client.ts	Singleton OpenAI; modelo efetivo; cálculo de custo USD
runtime-config.ts	Admin escolhe modelo; persiste em .controlaai/runtime.json
media-processor.ts	Whisper (áudio WhatsApp → texto); pdf-parse (extrato PDF)
logger.ts	Audita cada chamada em ai_logs (tokens, ms, operação, source)
category-resolver.ts	Aliases + inferência (pizza→Alimentação, uber→Transporte)

Consultas financeiras (insights.ts)

Responde perguntas em linguagem natural com dados reais do PostgreSQL:

queryType	Exemplo do usuário	O que calcula
monthly_spending	"Quanto gastei?"	Soma despesas do mês
top_spending_days	"Quais dias gastei mais?"	Agrupa por dia
biggest_expense	"Maior despesa?"	MAX amount
can_spend	"Posso gastar 500?"	Renda budget – gastos + projeção
health_check	"Situação financeira?"	KPIs agregados
month_comparison	"Comparei com mês passado"	Dois períodos
income_profile_status	"Já tenho renda cadastrada?"	budgets + settings

Também: generatePeriodReport (semanal/mensal/anual), KPIs do dashboard.

Persistência e contexto

Arquivo	Função detalhada
transaction-service.ts	INSERT transactions; formata resposta  ; lista categorias
financial-memory.ts	Preferências aprendidas em financial_memory (categorias, perfil renda)
user-context.ts	getUserFinancialContext — agrega renda, top categorias, flags onboarding

Entry serverless

Arquivo	Função
index.ts	Entry Vercel (sem Baileys) — apenas rotas API

Ligação WhatsApp ↔ IA (fluxo resumido)

```
Baileys (client.ts)
  → message-handler.ts
    → user-resolver.ts (telefone → user_id)
    → media-processor.ts (áudio/PDF, se mídia)
    → financial-agent.ts
      → onboarding-agent | goal-agent | income-classifier
      → parser.ts (OpenAI GPT)
      → transaction-service.ts | insights.ts (consultas)
    → whatsapp-bubbles.ts (resposta)
  → whatsapp_messages + ai_logs (persistência)
```

4. Fluxos principais

4.1 Mensagem WhatsApp → transação

[Diagrama Mermaid — ver PNGs em `./png/` e `./DATABASE_DIAGRAMAS.md`.]

Lógica:

1. Baileys recebe evento `messages.upsert`.
2. `jid-resolver` extrai telefone real (LID/PN + mapeamento local).
3. `user-resolver` — regra crítica: telefone da conversa → `users.id`; sem cadastro **não** registra nada.
4. `financial-agent` — pipeline unificado (WhatsApp + chat web):
 - Usuário novo → `onboarding-agent` (renda mensal + saldo em conta)
 - Pedido de meta → `goal-agent` (criação conversacional)
 - Transação/consulta → `parser` + `transaction-service` / `insights`
5. Áudio → Whisper; imagem → visão GPT; PDF → `pdf-parse` + `lote`.
6. Respostas com rodapé profissional via `app-links.ts` (só quando relevante).
7. Tudo em `whatsapp_messages` e `ai_logs`.

4.2 Login web → dashboard

1. `POST /auth/login` → valida `bcrypt` → emite JWT (7 dias).
2. Frontend guarda token → `Authorization: Bearer`.
3. `GET /api/transactions`, `/api/dashboard/summary` etc. usam `authPreHandler`.
4. Dashboard agrega receitas/despesas do mês via Drizzle.

4.3 Admin conecta WhatsApp

1. Admin faz login (`admin@admin.com`).
2. Acessa `/admin/whatsapp` → `GET /api/admin/whatsapp/status`.
3. `POST /api/admin/whatsapp/connect` → Baileys gera QR.
4. Admin escaneia QR no celular → credenciais em `.baileys-session/`.
5. Estado persistido em `whatsapp_connection` (`id = "main"`).
6. `keep-alive.ts` verifica conexão a cada 30 min.

4.4 Chat IA no painel web

1. `GET /api/ai/welcome` → boas-vindas ou onboarding se usuário novo.
2. `POST /api/ai/chat` → `processFinancialAgentMessage` (mesma lógica do WhatsApp).
3. Histórico em `ai_conversations.messages` (JSONB).

4.5 Perfil de renda (novos e existentes)

1. **Primeira informação de renda:** valor salvo em `budgets.total_income_expected` + defaults silenciosos em `user_settings` (`income_recurrence=manual`, `income_type=other`, `onboarding_completed=true`).

2. **Após salvar uma vez:** o agente **não** repete perguntas de tipo/recorrência/dia de pagamento; só reabre o fluxo se o usuário disser *configurar renda* ou similar.
3. **Usuário novo (modo full):** após informar renda, pode perguntar saldo em conta (opcional); demais campos do perfil não são obrigatórios.
4. **Usuário existente** sem renda no mês: pede valor **uma vez**; lembrete discreto só enquanto `budgets.total_income_expected` estiver vazio.
5. Disparadores de cadastro: *configurar renda*, valor isolado (4500) **somente se ainda não houver renda salva**.
6. Renda gravada em `budgets.total_income_expected` do mês corrente + `financial_memory.income_profile`.

4.6 Metas via WhatsApp

1. Usuário diz "quero criar uma meta" → `goal-agent` inicia fluxo.
2. Coleta tipo, nome, **valor** (`limit_amount/target_amount`) e **prazo** (`duration_months`, ex.: 5 meses, 1 ano = 12) → `INSERT goals`.
3. Parser estruturado em `api/goal-parser.ts` — separa valor de prazo (evita confundir "5 meses" com R\$ 5).
4. Progresso calculado em `goals-service.ts` a partir das transações.

5. Backend — servidor (`src/`)

5.1 `src/index.ts` — Boot

Passo	O que faz
1	Carrega <code>.env</code> via <code>env.ts</code>
2	<code>initRuntimeConfig()</code> — lê modelo OpenAI salvo em disco
3	Cria app Fastify + CORS
4	Registra <code>/health</code> (liveness para Railway)
5	Registra rotas: <code>auth</code> , <code>api</code> , <code>extended</code> , <code>whatsapp</code>
6	<code>ensureAdminUser()</code> — garante admin@admin.com
7	<code>initWhatsApp()</code> — inicia Baileys + keep-alive
8	Escuta porta PORT (padrão 3333)

5.2 `src/auth.ts` — Autenticação

- **Registro:** valida Zod → hash `bcrypt` (10 rounds) → insert `users` + `user_settings` → JWT.
- **Login:** busca por email → `bcrypt.compare` → JWT.
- **Middleware `authPreHandler`:** extrai Bearer token → `jwt.verify` → carrega usuário em `request.user`.

5.3 `src/api-routes.ts` — CRUD principal

Prefixo implícito `/api` (registrado no Fastify). Endpoints principais:

Método	Rota	Função
GET	<code>/transactions</code>	Lista transações do usuário
POST	<code>/transactions</code>	Cria lançamento manual
GET	<code>/categories</code>	Categorias globais + do usuário
GET	<code>/dashboard/summary</code>	Totais do mês
PUT	<code>/budgets/:month</code>	Orçamento mensal

5.4 `src/extended-routes.ts` — IA e metas

Método	Rota	Função
POST	<code>/ai/chat</code>	Chat conversacional
GET	<code>/ai/kpis</code>	Indicadores financeiros
GET	<code>/ai/insights</code>	Insights automáticos
CRUD	<code>/goals</code>	Metas financeiras
POST	<code>/imports/pdf</code>	Importação de extrato
GET	<code>/whatsapp/conversations</code>	Histórico do usuário
Admin	<code>/admin/ai/*</code>	Logs IA, troca de modelo

5.5 `src/goals-service.ts`

Calcula progresso real de cada meta somando transações do período. Metas de **poupança** com `duration_months` usam janela [`created_at`, `deadline_at`]; metas de **limite** usam ciclo mensal/trimestral/anual (`period_type`).

5.6 `src/db/index.ts`

- Cliente postgres com pool (max 10).
 - SSL automático para Neon.
 - `prepare: false` quando usa pooler Neon.
 - Exporta `db` (Drizzle) usado em todo o projeto.
-

6. OpenAI (api/)

Arquivo	Responsabilidade
financial-agent.ts	Orquestrador — onboarding, metas, parser, transações, consultas
onboarding-agent.ts	Rapport renda mensal + saldo (usuários novos)
goal-agent.ts	Criação conversacional de metas
app-links.ts	URLs do painel + rodapés WhatsApp/chat
index.ts	Entry serverless Vercel (sem WhatsApp)
openai-client.ts	Singleton OpenAI, modelo efetivo, custo USD
runtime-config.ts	Admin escolhe modelo; persiste em <code>.controlaai/runtime.json</code>
prompts.ts	System prompts do ControlaAI (parser, chat, documentos)
parser.ts	Extraí FinancialIntent JSON de texto/imagem/PDF
category-resolver.ts	Normaliza categorias + inferência por descrição
insights.ts	Chat, KPIs, relatórios, consultas ("quanto gastei?")
transaction-service.ts	Persiste transação a partir do intent
financial-memory.ts	Aprende categorias preferidas por usuário
media-processor.ts	Whisper (áudio) + pdf-parse (extrato)
logger.ts	Grava cada chamada em <code>ai_logs</code>

Formato FinancialIntent (parser)

```
{
  "intent": "transaction | query | report | goal | unknown",
  "type": "expense | income | transfer",
  "value": 50.00,
  "category": "Alimentação",
  "description": "mercado",
  "date": "2026-06-06",
  "queryType": "monthly_spending"
}
```

Fallback: se `OPENAI_API_KEY` ausente, `parser.ts` usa regex local (`parseLocalIntent`).

7. WhatsApp / Baileys (whatsapp/)

Arquivo	Responsabilidade
client.ts	Socket Baileys, QR, reconexão, envio de texto
session-utils.ts	Pasta .baileys-session, creds.json
message-handler.ts	Pipeline mensagem → agente financeiro → banco (bolhas)
whatsapp-bubbles.ts	Envio de respostas em múltiplas mensagens (conversa humanizada)
inbound-reply-guard.ts	Bloqueia envio outbound sem inbound em processamento
message-dedup.ts	Deduplica messageId — evita replay na reconexão Baileys
user-resolver.ts	Telefone → user_id (obrigatório; variantes BR)
jid-resolver.ts	Resolve LID/PN + arquivo lid-mapping*_reverse.json
keep-alive.ts	Timer 30 min — health check + reconexão
routes.ts	API admin /api/admin/whatsapp/*
baileys-log.ts	Buffer circular de logs (500 entradas)

Sessão Baileys

- **Local:** backend/.baileys-session/ (padrão quando BAILEYS_SESSION_DIR vazio).
- **Produção (Railway/Docker):** volume em /data/.baileys-session.
- Arquivo chave: creds.json com "registered": true após QR.

Keep-alive (30 min)

1. Ignora se não há sessão pareada.
2. Verifica socket (sock.user existe?).
3. Se saudável → refresh preventivo opcional.
4. Se offline → reconecta com useMultiFileAuthState.

Fluxo conversacional WhatsApp (agente IA)

1. **Telefone não cadastrado** → envia bolhas com link de registro (buildRegistrationBubbles).
2. **Pós-cadastro** (telefone estava pendente ou conta nova) → parabéns + sugestões de meta (buildPostRegistrationBubbles).
3. **Meta definida** → convite para registrar gastos/receitas em bolhas.
4. **Usuário cadastrado** → saudação humanizada pedindo gastos; suporta texto, áudio, comprovante, PDF.
5. **Capacidades:** registrar ganhos/gastos, análises, projeções, relatórios.
6. **Anti-repetição:** conversation-history.ts consulta whatsapp_messages outbound recentes.
7. **Histórico:** todas as mensagens inbound/outbound persistidas em whatsapp_messages.
8. **Sem envio proativo:** sendToChat só funciona dentro de runWithInboundReply (resposta a inbound real).

8. Frontend (frontend/)

Rota	Página	Função
/	Dashboard	KPIs, gráficos, transações
/login, /register	Auth	JWT usuário comum
/admin/login	AdminLogin	JWT exclusivo admin
/goals	Goals	Metas financeiras
/ai	AiChat	Chat IA (histórico interno na sidebar)
/settings	Settings	Perfil, tema, export CSV
/admin/whatsapp	WhatsApp	QR Baileys, modelo OpenAI (admin)
/admin/ai-logs	AiLogs	Logs OpenAI (admin)
*	NotFound	404

Cliente HTTP: frontend/src/lib/api.ts — todas as chamadas REST.

Autenticação: frontend/src/lib/auth.tsx — JWT em localStorage.

Mapa de arquivos: frontend/src/MAPA-SISTEMA.tsx — catálogo completo da aplicação.

Documentação no código: cabeçalho Doc TCC: TCC_DOCUMENTACAO.md + comentários em português nos arquivos de aplicação (exclui components/ui/* shadcn).

Variável VITE_API_URL aponta para o backend (dev: proxy Vite → porta 3333).

9. Arquitetura de banco de dados

SGBD: PostgreSQL 15+ (Neon serverless)

ORM: Drizzle

Banco oficial: controlaai

9.1 Diagrama entidade-relacionamento

[Diagrama Mermaid — ver PNGs em ./png/ e ./DATABASE_DIAGRAMAS.md.]

9.2 Enums PostgreSQL

Enum	Valores
plan	free, pro, premium
category_type	expense, income
transaction_type	expense, income
transaction_source	whatsapp, web, recurring, manual
goal_period	monthly, quarterly, yearly
goal_kind	limit, saving
whatsapp_connection_status	disconnected, connecting, qr, connected, error
whatsapp_message_direction	inbound, outbound
whatsapp_message_type	text, audio, image, document, video, other
ai_log_status	success, error, pending
import_status	pending, processing, completed, failed

9.3 Tabelas — detalhamento

users

Conta do usuário. Criada via web (email/senha) ou automaticamente via WhatsApp (wa_5511999999999@whatsapp.controla.ai).

Coluna	Tipo	Descrição
id	UUID PK	Identificador
name	text	Nome exibido
email	text UNIQUE	Login web
password_hash	text	bcrypt
phone	text UNIQUE	Vínculo WhatsApp (55DDD...)
plan	enum	free / pro / premium

user_settings

Preferências 1:1 com usuário.

Coluna	Descrição
alert_at_80 / alert_at_100	Alertas de meta
theme_preference	Tema UI
onboarding_completed	Rapport inicial concluído
initial_balance	Saldo em conta informado no onboarding
income_recurrence	monthly_fixed manual weekly — memória do agente

categories

Categorias globais (`user_id` NULL) + personalizadas por usuário. Campos: name, icon, type, color, is_default.

transactions

Núcleo financeiro — cada gasto ou receita.

Coluna	Tipo	Descrição
amount	numeric(12,2)	Valor BRL
type	enum	expense / income
source	enum	whatsapp / web / manual
raw_message	text	Texto original (WhatsApp)
occurred_at	timestamp	Data do lançamento

goals + goal_checkpoints

Metas por categoria/período. Colunas principais: `limit_amount` (valor/teto), `target_amount` (alvo poupança), `duration_months` (prazo em meses, ex. 5 ou 12), `deadline_at` (data alvo calculada), `period_type` (monthly/quarterly/yearly). Checkpoints guardam snapshot mensal (spent, limit, percentage, exceeded).

budgets

Orçamento mensal único por usuário/mês (`UNIQUE user_id + month`).

whatsapp_connection

Singleton (`id = 'main'`) — estado da conexão Baileys do número oficial.

whatsapp_messages

Log de todas as mensagens (inbound/outbound) com vínculo opcional a `transaction_id`.

Índices: `remote_phone`, `created_at`, `user_id`.

ai_logs

Auditoria de cada chamada OpenAI (tokens, custo USD, operação, source).

financial_memory

Preferências aprendidas (preference_key + JSON) — ex.: categorias mais usadas.

ai_conversations

Histórico do chat web em JSONB (messages array).

document_imports

Rastreo de PDFs importados pelo painel.

9.4 Diagramas e dados atuais (export TCC)

Artefatos em ./ — gerados por npm run db:export-tcc:

Arquivo	Conteúdo
TCC_DOCUMENTACAO.pdf	Documentação completa em PDF
TCC_DOCUMENTACAO.md	Cópia Markdown do documento oficial
TCC_DOCUMENTACAO.txt	Versão texto plano (sem formatação MD)
DATABASE_DIAGRAMAS.md	Índice visual com todos os PNGs embutidos
DATABASE_SNAPSHOT.md	Colunas + dados atuais (senhas mascaradas)
CONEXOES_BANCO_DADOS.md	Conexões e FK — mapa, Mermaid, lista das 18 ligações, entradas/saídas por tabela
ARQUITETURA_BANCO_COMPLETA.md	Arquitetura MD completa — Mermaid ER, FK, colunas, PK
png/arquitetura-banco-diagrama.png	2900px — linhas curtas vizinho-a-vizinho (sem atravessar o diagrama) + hub users.id
png/arquitetura-banco-detallhes.png	Diagrama simplificado + tabela completa das 18 FK
png/database-arquitetura-completa.png	Legado — coluna única 1920px
png/00-visao-geral.png	Visão geral dos 5 domínios
png/grupo-core.png	users, user_settings, categories, transactions, budgets
png/grupo-metas.png	goals, goal_checkpoints
png/grupo-whatsapp.png	whatsapp_connection, whatsapp_messages, whatsapp_sessions
png/grupo-ia.png	ai_logs, ai_conversations, financial_memory, document_imports
png/grupo-outros.png	recurring_transactions, subscriptions
png/tabela-<nome>.png	16 PNGs — uma tabela cada, com todas as colunas

O ERD único com tudo junto ficava ilegível ao dar zoom. Por isso os diagramas foram divididos por **domínio** e por **tabela**.

```
cd backend
npm run db:export-tcc # PNGs + snapshot + arquitetura completa
npm run tcc:docs      # TXT + PDF atualizados
```

9.5 Comandos de banco

```
cd backend
npm run db:push      # Aplica schema Drizzle no Neon
npm run db:seed      # Categorias padrão (se vazio)
npm run db:setup     # push + seed
npm run db:check     # Testa conexão
```

10. Autenticação e segurança

Mecanismo	Implementação
Senhas	bcrypt, 10 salt rounds
Sessão web	JWT HS256, expira em 7 dias
Rotas protegidas	authPreHandler — Bearer obrigatório
Admin	Apenas admin@admin.com — adminPreHandler
WhatsApp admin	QR/connect/logs só para admin
Sessão Baileys	Arquivos locais, fora do git
CORS	Origins do FRONTEND_URL + localhost

11. Variáveis de ambiente

Arquivo: backend/.env (ver .env.example)

Variável	Obrigatória	Descrição
DATABASE_URL	Sim	PostgreSQL (Railway ou Neon)
JWT_SECRET	Sim	Chave JWT
OPENAI_API_KEY	Recomendada	Parser e chat IA
OPENAI_MODEL	Não	Padrão: gpt-4o-mini
FRONTEND_URL	Sim (prod)	CORS + links nas mensagens IA
REGISTER_URL	Não	Link cadastro WhatsApp
PORT	Não	Padrão 3333
BAILEYS_SESSION_DIR	Não	Padrão: backend/.baileys-session
ENABLE_WHATSAPP	Não	false desliga Baileys
WHATSAPP_KEEPLIVE_INTERVAL_MS	Não	Padrão 1800000 (30 min)

12. Deploy e execução

Local

```
cd backend
npm install
npm run db:push
npm run dev          # tsx watch src/index.ts

cd ../frontend
npm install
npm run dev          # http://localhost:5173
```

Produção

Componente	Plataforma	Entry
Backend + WhatsApp	Railway	node dist/src/index.js
Frontend	Vercel	https://controlaai-frontend.vercel.app
Banco	Railway PostgreSQL	DATABASE_URL

Migration onboarding: `npm run db:migrate:onboarding` OU `drizzle/0001_onboarding_settings.sql`.

13. Mapa de arquivos comentados

Cada arquivo abaixo possui **comentários em português** no código-fonte (cabeçalho JSDoc Doc TCC: TCC_DOCUMENTACAO.md + imports, hooks, API e seções de UI).

Backend ([backend/](#) — ver [src/MAPA-SISTEMA.ts](#))

Pasta	Arquivos comentados (PT)
src/	index, env, auth, api-routes, extended-routes, goals-service, db/index, db/schema, db/ensure-admin, utils/*
api/	financial-agent, onboarding-agent, goal-agent, goal-parser , app-links, parser, prompts, transaction-service, category-resolver, insights, financial-memory, media-processor, openai-client, runtime-config, logger, index
whatsapp/	client, message-handler, user-resolver, jid-resolver, routes, session-utils, keep-alive, baileys-log

Lista exportada: BACKEND_APPLICATION_FILES em [backend/src/MAPA-SISTEMA.ts](#).

Frontend ([frontend/src/](#) — aplicação, exceto [components/ui/*](#))

Pasta	Arquivos
Raiz	main.tsx, App.tsx, MAPA-SISTEMA.tsx
lib/	api.ts, auth.tsx, routes.ts, admin.ts, utils.ts, chart-colors.ts, category-icons.tsx, mockData.ts
pages/	Dashboard.tsx, Goals.tsx, AiChat.tsx, Settings.tsx, Login.tsx, AdminLogin.tsx, Register.tsx, WhatsApp.tsx, AiLogs.tsx, Index.tsx, NotFound.tsx
components/	Layout.tsx, DashboardDialogs.tsx, NavLink.tsx, RequireAdmin.tsx, RequireAdminAuth.tsx, ChartPlotArea.tsx, Logo.tsx, AppErrorBoundary.tsx
hooks/	use-capabilities.ts, use-mobile.tsx, use-toast.ts

Catálogo exportado em MAPA-SISTEMA.tsx (FRONTEND_APPLICATION_FILES). Ao criar ou renomear arquivos, adicionar comentários e **atualizar esta seção**.

14. Histórico de alterações

Data	Versão	Alteração
jun/2026	2.0	Reorganização TCC: <code>api/</code> (OpenAI), <code>whatsapp/</code> (Baileys), achatamento de <code>src/modules/</code>
jun/2026	2.0	Criação deste documento único <code>TCC_DOCUMENTACAO.md</code>
jun/2026	2.1	Comentários linha a linha em português nos arquivos principais
jun/2026	2.1	Regra Cursor <code>.cursor/rules/tcc-documentacao.mdc</code> para manter doc sincronizada
jun/2026	2.2	Comentários TCC (cabeçalho JSDoc + inline PT) em todos os 15 arquivos <code>backend/api/*.ts</code>
jun/2026	2.2	Comentários TCC completos em <code>backend/src/</code> (exc. MAPA-SISTEMA, seeds) e <code>backend/whatsapp/*.ts</code>
jun/2026	2.3	Documentação TCC no frontend: 36 arquivos de aplicação
jun/2026	3.0	Agente unificado, onboarding, metas WhatsApp, app-links, phone BR, Railway
jun/2026	3.0	Comentários linha a linha em todo backend de negócio + frontend aplicação
jun/2026	3.0	MAPA-SISTEMA.ts / MAPA-SISTEMA.tsx catálogos completos; migration 0001 onboarding
jun/2026	3.1	Perfil de renda para usuários existentes; recorrência (fixa/manual/semanal); migration 0002
jun/2026	3.2	Agente WhatsApp humanizado: bolhas, fluxo cadastro→meta→gastos, anti-repetição; favicon
jun/2026	3.3	Guarda inbound obrigatório + dedup messageId — sem disparo sem mensagem recebida
jun/2026	3.4	Renda persistida incrementalmente (budgets + user_settings + financial_memory); lembrete não repete valor já informado
jun/2026	3.5	Link do painel após registrar renda (<code>buildDashboardReportBubbles</code>); Dashboard mobile + gráficos com dados reais corrigidos
jun/2026	3.6	Parser de metas (<code>goal-parser.ts</code>): valor → <code>limit_amount/target_amount</code> , prazo → <code>duration_months + deadline_at</code> ; migration <code>0004_goal_duration_months.sql</code> ; script <code>npm run db:migrate:goal-duration</code>
jun/2026	3.7	Agente WhatsApp: <code>parseMoneyAmount</code> (5k/5mil/5000), histórico no parser, categoria por descrição, renda não intercepta transações (<code>transaction-intent.ts</code>)
jun/2026	3.8	Renda vs ganho: <code>income-classifier.ts</code> + <code>user-context.ts</code> ; perfil salário/freela; migration <code>0005</code>
jun/2026	3.9	Saldo projetado com renda do budget; categorias via IA; consultas (dias/ renda); mensagens ricas pós-registro (<code>assistant-response.ts</code>); link público Vercel
jun/2026	4.0	Removidas páginas <code>/conversations</code> e <code>/integrations</code> ; Layout mobile com nav scrollável; chat IA full-height no celular
jun/2026	4.1	Fix WhatsApp: saudações (Olá) priorizadas antes do onboarding — não retorna mais "Valor não identificado"
jun/2026	4.2	<code>message-text.ts</code> : normalização inbound; saudação limpa sessão onboarding; welcome web não pré-cria sessão; passo income re-pergunta em vez de erro
jun/2026	4.3	Boas-vindas completas (gastos/ganhos/metast/análises); fallback sempre menu; reinício obrigatório do backend; handler WA limpa sessão em saudação

jun/2026	4.4	WhatsApp: saudação responde direto no message-handler (sem passar onboarding); matar processo na porta 3333 antes de <code>npm run dev</code>
jun/2026	4.5	Anti-replay: ignora mensagens antigas pós-QR/reconnect; limpa sessões ao cumprimentar; fallback sempre menu; histórico parser 10 msgs
jun/2026	4.6	Renda salva uma vez: <code>saveIncomeProfileOnce</code> , <code>hasMonthlyIncomeSaved</code> ; sem repetir tipo/recorrência após 1º save; lembrete só sem renda; export TCC (<code>./*.png</code> , <code>DATABASE_SNAPSHOT.md</code> , PDF); §15 referência completa
jun/2026	4.7	Pasta <code>./</code> na raiz: MD, PDF, PNGs ERD e snapshot do banco; scripts <code>db:export-tcc</code> e <code>tcc:pdf</code>
jun/2026	4.8	Frontend: logo → Dashboard; perfil → Configurações/#renda-mensal; títulos dinâmicos das abas; seção renda em Settings
jun/2026	4.9	Doc §3.1 mapa detalhado whatsapp/ + api/ (IA, consultas, Baileys); PNGs do banco divididos por domínio e por tabela em <code>./png/</code>
jun/2026	5.0	Export TXT: <code>TCC_DOCUMENTACAO.txt</code> + script <code>npm run tcc:txt</code> ; formatos MD + TXT + PDF
jun/2026	5.1	Guia rápido de pastas no início do doc; PNG <code>database-arquitetura-completa.png</code> (HTML 1920px coluna única)
jun/2026	5.2	PNG arquitetura refeito: coluna única, fontes grandes (30–40px), 1920px largura — legível sem zoom extremo
jun/2026	5.3	<code>ARQUITETURA_BANCO_COMPLETA.md</code> + PNG diagrama 16:9 com ligações SVG + PNG detalhes colunas/FK
jun/2026	5.4	PNG detalhes inclui diagrama visual com linhas de conexão entre tabelas no topo
jun/2026	5.5	<code>CONEXOES_BANCO_DADOS.md</code> — documento dedicado às ligações FK com mapa, Mermaid e entradas/saídas por tabela
jun/2026	5.6	PNGs refeitos: tabelas largas com todas as colunas, linhas FK ortogonais coloridas e barramento em users
jun/2026	5.7	Layout original 4 zonas restaurado; linhas FK em corredores paralelos sem cruzamento
jun/2026	5.8	Posições fixas espalhadas (layout original) + colunas completas nas caixas
jun/2026	5.9	Diagrama 4 colunas verticais + linhas em calhas (sem cruzamento) + numeração 1–18
jun/2026	6.0	Diagrama simplificado: 6 linhas A–F + hub <code>users.id</code> ; tabelas mais espaçadas; 18 FK na tabela do PNG detalhes
jun/2026	6.1	Linhas curtas só entre vizinhos (lado/abaixo); Núcleo+Metas adjacentes; <code>whatsapp_messages</code> ao lado de <code>transactions</code>
jun/2026	6.2	Remove bordas pontilhadas das zonas; setas FK mais grossas e visíveis (12px + rótulos maiores)
jun/2026	6.3	Setas retas só no vão entre vizinhos (horizontal/vertical); sem calhas laterais
jun/2026	6.4	18 FK visíveis — linhas na margem/corredor entre colunas; SVG sobre tabelas; <code>users.id</code> em azul claro
jun/2026	6.5	Remove 18 linhas sobrepostas; só 6 setas A–F com calha exclusiva; <code>users.id</code> na caixa azul
jun/2026	6.6	Setas só na margem externa; rótulos HTML fora das tabelas; SVG atrás das caixas

15. Referência completa — agente IA, módulos e lógicas

Documento único do TCC. Esta seção consolida todas as lógicas implementadas, usos de IA e funções criadas. Complementa as seções 4–8 com detalhe de implementação.

15.1 Pipeline unificado (financial-agent.ts)

WhatsApp (message-handler.ts) e chat web (extended-routes.ts) chamam processAgentMessage(userId, text) — mesma ordem de prioridade:

[Diagrama Mermaid — ver PNGs em ./png/ e ./DATABASE_DIAGRAMAS.md.]

Etap	Condição	Módulo	Resultado
1	isGreetingMessage	message-text.ts	Menu welcome — nunca pede valor
2	isJustRegistered	conversation-context.ts	Bolhas pós-cadastro
3	hasIncomeClarifySession	income-classifier.ts	Pergunta 1=renda / 2=ganho
4	isIncomeMessage sem expense	income-classifier.ts	Roteia renda mensal vs ganho pontual
5	Sessão onboarding ou trigger renda	onboarding-agent.ts	Só se não tem renda salva
6	Acknowledgment (ok, beleza)	financial-agent.ts	Pede renda só se needsIncomeProfile
7	Meta	goal-agent.ts	Fluxo valor → prazo → INSERT goals
8	Parser	parser.ts	GPT-4o-mini + fallback regex
9	Fallback	financial-agent.ts	Menu welcome (não erro seco)

Anti-repetição: finalizeResponse → ensureUniqueResponse + appendDashboardIfIncomeJustSaved.

15.2 Regra de renda mensal (v4.6)

Regra	Implementação
Renda informada uma vez → salva no banco	saveMonthlyIncome → budgets.total_income_expected + financial_memory + user_settings
Não repetir perguntas de tipo/recorrência/dia	saveIncomeProfileOnce grava defaults: incomeRecurrence>manual, incomeType=other, onboardingCompleted=true
Detectar renda já salva	hasMonthlyIncomeSaved(userId) consulta budgets do mês atual
Lembrete opcional	buildIncomeProfileReminder — string vazia se renda já existe
Reconfigurar	Usuário diz <i>configurar renda / minha renda</i> → isIncomeProfileTrigger reabre fluxo
Valor isolado (4500)	Salva direto via saveIncomeProfileOnce — sem multi-etapas
Clarificação ambígua	Escolha 1 (renda) → saveIncomeProfileOnce com valor da sessão

Funções exportadas (onboarding-agent.ts):

- hasMonthlyIncomeSaved, needsIncomeProfile, needsProfileSetup
- saveIncomeProfileOnce, processOnboardingAgentMessage
- buildIncomeProfileReminder, getOnboardingWelcomeIfNeeded
- flushOnboardingSessionToDb, clearOnboardingSession

15.3 Renda vs ganho pontual (income-classifier.ts)

Rota	Exemplo	Ação
profile_setup	"Recebo 5000", "meu salário é 4500"	Salva perfil mensal
one_time_gain	"Ganhei 200 de freela", "recebi 150"	INSERT transaction type=income
ambiguous	"5000" sem contexto	Pergunta 1=renda / 2=ganho
not_income	Demais mensagens	Delega ao agente principal

classifyIncomeMessage usa regex + contexto de user-context.ts (UserFinancialContext).

15.4 Parser OpenAI (parser.ts + prompts.ts)

Operação	Modelo	Entrada	Saída
parse	gpt-4o-mini	Texto + histórico 10 msgs + categorias	FinancialIntent JSON
transcribe	whisper-1	Áudio WhatsApp	Texto → parse
vision	gpt-4o-mini	Imagem nota fiscal	Texto → parse
document	gpt-4o-mini	PDF extraído	Texto → parse
chat	gpt-4o-mini	Chat web multi-turn	Resposta natural

Schema Zod FinancialIntent:

```
{ intent: "transaction"|"query"|"report"|"goal"|"unknown",
  type?: "expense"|"income", value?: number, category?: string,
  description?: string, queryType?: string }
```

Fallback local (parseLocalIntent): regex para consultas, metas, gastos/receitas quando OpenAI indisponível.

Auditoria: cada chamada → ai_logs via logger.ts (tokens, custo USD, ms).

15.5 Consultas financeiras (`insights.ts`)

queryType	Pergunta exemplo	Dados usados
monthly_spending	Quanto gastei?	SUM transactions expense mês
top_spending_days	Quais dias gastei mais?	GROUP BY day
biggest_expense	Maior despesa?	MAX amount
can_spend	Posso gastar 500?	budget – gastos + projeção renda
health_check	Situação financeira?	KPIs agregados
month_comparison	Comparei com mês passado	Dois períodos
income_profile_status	Já tenho renda?	budgets + user_settings

Saldo projetado: renda de `budgets.total_income_expected` – gastos acumulados + dias restantes.

15.6 Metas (`goal-agent.ts` + `goal-parser.ts`)

Fluxo conversacional WhatsApp:

1. Usuário: *Quero juntar 5000 em 6 meses*
2. `goal-parser.ts` extrai `targetAmount`, `durationMonths`, `deadlineAt`
3. INSERT `goals` (`goal_type=saving`, `duration_months`, `deadline_at`)
4. Resposta com progresso + link painel

Migration 0004: colunas `duration_months`, `deadline_at` em `goals`.

15.7 Transações (`transaction-service.ts` + `category-resolver.ts`)

Função	Descrição
<code>createTransactionFromIntent</code>	Resolve categoria, INSERT <code>transactions</code> , formata resposta 
<code>listAvailableCategories</code>	Globais + personalizadas do usuário
<code>inferCategoryFromDescription</code>	pizza→Alimentação, uber→Transporte (alias + IA)
<code>resolveCategoryId</code>	UUID da categoria ou cria personalizada

`parseMoneyAmount` (`money.ts`): 5k, 5 mil, R\$ 30,00, 4500.

15.8 Contexto do usuário (`user-context.ts`)

`getUserFinancialContext(userId)` agrega:

- Perfil renda: `monthlyAmount`, `recurrence`, `incomeType`, `payDay`
- Top categorias (`financial_memory`)
- Contagem transações, fase conversa
- Flags onboarding

Usado por parser, income-classifier e insights.

15.9 WhatsApp (whatsapp/)

Arquivo	Responsabilidade
client.ts	Baileys socket, QR, reconexão, anti-replay (>4 min ou 20s pós-conexão)
message-handler.ts	Saudação direta → limpa sessões → processAgentMessage → bolhas
whatsapp-bubbles.ts	Split `
inbound-reply-guard.ts	Só responde após inbound real
message-dedup.ts	Dedup por whatsappMessageId
user-resolver.ts	Telefone → user_id (auto-create se novo)
jid-resolver.ts	LID/PN mapping

15.10 Memória e conversa

Módulo	Armazenamento	Uso
financial-memory.ts	financial_memory JSONB	Categorias preferidas, perfil renda
conversation-context.ts	Memória RAM	Fase: goals/expenses, justRegistered
conversation-history.ts	whatsapp_messages	Últimas outbound, histórico parser
assistant-response.ts	—	Templates ricos pós-transação

15.11 Frontend (v4.0)

Rotas ativas: /, /dashboard, /goals, /ai-chat, /settings, /admin/*.

Removidas: /conversations, /integrations.

Layout mobile: nav horizontal scrollável; chat IA 100dvh no celular.

15.12 Variáveis OpenAI e custos

Env	Default	Uso
OPENAI_API_KEY	—	Obrigatória para parser/chat
OPENAI_MODEL	gpt-4o-mini	Parser + chat
Admin runtime	.controlaai/runtime.json	Override modelo pelo painel

Custo estimado por request em ai_logs.cost_usd.

15.13 Mapa de funções por arquivo

► **backend/api/** (clique para expandir)

- ▶ **backend/whatsapp/**
- ▶ **backend/src/**

15.14 Dados atuais do banco

Ver: [./DATABASE_SNAPSHOT.md](#) · [./DATABASE_DIAGRAMAS.md](#)

Diagramas PNG (legíveis):

- Visão geral: `./png/00-visao-geral.png`
- Por domínio: `grupo-core`, `grupo-metas`, `grupo-whatsapp`, `grupo-ia`, `grupo-outros`
- Por tabela: `./png/tabela-<nome>.png` (16 arquivos)

Fim do documento. Mantenha este arquivo sincronizado com o código.